©VIT IPR&TTCELL

VIT Vellore Institute of Technology (Deemed to be University under section 3 of UGC Act, 1956)	**Invention Disclosure Format (IDF)-B**	Document No. 02-IPR-R003 Issue No/Date 2/01.02.2024 Amd. No/Date 0/00.00.0000

1. Title of the invention:

System and Method for Anticipatory, Confidence-Gated Reservation of Scheduling Priority for Not-Yet-Created Processes Using Learned Event-to-Process Latency

2. Field /Area of invention:

The invention is mainly related to Operating Systems, with focus on CPU Scheduling, Process Management and Computer Resource Control. It deals with predicting the creation of a process from an earlier user action, and preparing a time-bounded scheduling-priority reservation for that process before it exists. In the described embodiment the user action is saving a source file and the predicted process is a software build, which makes the invention applicable to software-development environments; the same mechanism applies to other workflows in which a user action reliably precedes the creation of a process, such as running tests or rendering jobs. The invention covers learning of event-to-process latency, confidence-gated and contention-gated arming of a reservation, identity-bound claiming of a newly created process, and self-measurement of the benefit obtained. It is designed for Linux, using filesystem event monitoring (inotify), process-creation detection (netlink process connector or /proc), CPU-pressure information and control groups (cgroup v2). It is intended to improve the responsiveness of builds on resource-constrained and shared machines without modifying the operating-system kernel.

3. Prior Patents and Publications from literature

Patent/Publication	Description	Relevance to the Invention
Resource management based on device-specific or user-specific resource usage profiles (US10182018)	Monitors a user's and a device's resource usage and uses the resulting profiles to predict the resource needs of an application. It can allocate resources, such as memory, to an application before the user attempts to launch it, when the user is likely to launch it at a particular time, place or event, and can decide whether a launch is allowed.	Closest known predictive allocation of resources ahead of a process start. It works at application level from usage profiles (time, place, event) and prepares resources such as memory. It does not describe our learned latency between a specific upstream event in one subsystem and process creation in another, a time-windowed scheduling-priority reservation prepared as an empty group, identity-bound atomic claiming, or expiry of the reservation.
Predictive pre-launch for applications (US9508040B2)	Associates with each application a prediction of when a user may activate it, monitors system resources (such as CPU, GPU, memory, battery and I/O), and pre-launches an application when pre-launch rules based on resource availability and on the prediction are satisfied.	Closest known prediction-driven action before a process exists, with a resource-availability condition comparable to our contention gate. It pre-launches the application itself from usage-time predictions. It does not describe our learned latency from a specific upstream event to process creation, a scheduling-priority reservation prepared before the process exists, identity-bound claiming of the process that appears, or automatic expiry.

©VIT IPR&TTCELL

Patent/Publication	Description	Relevance to the Invention
Prediction based priority scheduling (US8185899B2; continuation US8448178B2)	Predicts, from a historical log of task-request submissions, when high-priority requests are likely, and schedules or delays lower-priority requests that would interfere. Requests are held in priority queues and scheduled by rules, including thresholds on queue contents.	Closest known prediction-driven priority scheduling. It predicts demand for requests entering existing queues and withholds lower-priority work to avoid interference. It does not describe our prediction of the creation of a new process from an event in another subsystem, a reservation made before the process exists, or binding of a reservation to that process by identity.
Managing Scheduling of Processes (US20120185862A1, granted as US9009717B2)	Dynamically changes the base priority of a set of processes, and optionally their spawned child processes, according to monitored CPU and I/O-wait usage and a priority class assigned at start-up, which can be associated with an executable name in configuration.	Closest known work on automatic priority control with name-based classes and child-process coverage. Priority is adjusted for processes that already exist, from measured usage. It does not describe our learned delay between a user event and a future process, a reservation created before the process exists, or a time window with automatic expiry.
Anticipatory scheduling: A disk scheduling framework to overcome deceptive idleness in synchronous I/O (Iyer and Druschel, ACM SOSP 2001)	After serving a request, the disk scheduler may keep the disk idle for a short interval in anticipation of the next request of the same process, weighing the cost of idling against the expected benefit (a non-work-conserving discipline).	Closest known "anticipation" concept. It anticipates the next request of a process that already exists, within one resource (the disk). It does not describe our prediction of a new process from an event in another subsystem, a reservation of CPU scheduling priority, or binding to a future process.
Managing computing resource consumption of software applications using control groups to facilitate safety compliance (US 12,705,160)	On detecting an interference event between a user application and a functional safety application, determines the share of CPU, memory and disk I/O needed by the safety application and a separate share for the other application, using control groups.	Relevant as cgroup-based reservation of CPU, memory and I/O. It is triggered reactively by an interference event for an existing application. It does not describe our prediction of a future process, a learned latency, or expiry of a reservation created in advance.

Why the combination is not an obvious one. Predictive pre-launch and predictive resource allocation (US9508040B2, US10182018) predict when a user will start an application, from usage patterns such as time of day, place or event, and then pre-launch it or prepare resources for it; the prediction concerns the application, not a latency between two specific events in different subsystems, and the preparation is of the application or its memory, not of a scheduling-priority arrangement bound to the process that later appears. The closest priority-control art adjusts priority after a process has started, from measured usage (US20120185862A1). Prediction-based priority scheduling (US8185899B2) predicts demand for requests entering existing queues in order to delay lower-priority work, and anticipatory scheduling anticipates the next request of a process that already exists, within a single resource. Cgroup reservation art (US 12,705,160) reserves resources reactively after an interference event. In the references reviewed, none describes learning the latency between an event in one subsystem (the filesystem) and the creation of a process in another, creating a reservation that is not attached to any process, and binding it later by identity within a time window that expires. A combination would also have to establish that such a latency is stable enough to exploit; the invention addresses this with a confidence gate that abstains when it is not, and with the validation protocol of Section 8.

4. Summary and background of the invention

Software builds sit on a developer's critical path. A build is usually preceded by a file save, and on many projects the delay between the save and the start of the build process is stable enough to learn. Conventional operating-system schedulers assign priority only after a process has been created, using its observed behaviour, its parent's settings or administrator rules. During start-up, the build therefore competes with every other task at whatever priority it inherits, and the operating system has no means of protecting it in advance. Predictive approaches in adjacent fields, such as thermal management, data prefetching, power management and adaptive algorithm selection, act on existing processes, on data, or on hardware and power state.

Technical problem. A process started by a predictable user action begins execution without any scheduling advantage, because the scheduler can act on a process only after it exists. Reactive adjustment, which depends on measured usage or foreground state, leaves this start-up interval unprotected.

Technical solution. The proposed system observes upstream events (in the embodiment, file saves) in registered project directories and performs the following steps:

- **Latency Learning:** It learns, per project, class of saved file and build-tool profile, the delay between the last event and the creation of a matching build process.
- **Confidence and Contention Gating:** It predicts only when the learned delay is stable enough, and arms a reservation only when CPU contention is present and the expected benefit exceeds the expected cost; otherwise it abstains and default scheduling applies.
- **Anticipatory Reservation:** It prepares a time-windowed scheduling-priority reservation before the build process exists.
- **Identity-Bound Claim:** When a process matching the expected tool and project appears, it is moved into the reservation, and its descendants inherit it.
- **Expiry and Feedback:** Unused reservations are released automatically and every outcome is fed back into the model.
- **Self-Measured Benefit:** It withholds a small random share of reservations and compares build times at matched load, to estimate its own benefit and adapt.

Technical effect. The effect is a shorter unprotected start-up interval and a higher relative CPU share for the target process under contention, obtained with bounded impact on other processes because every reservation is time-limited, weight-capped and released automatically. The reservation is a time-bounded scheduling-priority reservation, that is, a raised relative CPU share under contention; it does not guarantee a fixed amount of CPU capacity. The magnitude of the effect is to be established by the validation protocol of Section 8.

The core innovation lies in combining a learned association between an upstream user event and a future process, a pre-existence time-windowed reservation bound to that process by identity, and a closed loop of gating, expiry, abstention and self-measured benefit. In the prior art reviewed, no single reference combined these elements. Assigning priority classes to processes by executable name at start-up, adjusting priority from measured usage, predicting data access, predicting future high-priority requests in order to delay lower-priority work, anticipating the next request of a running process, learning user activity patterns to prepare a device in advance for power management, and predicting when an application will be launched in order to pre-launch it or to allocate resources to it in advance are each known. The invention differs in that the reservation is created before the process exists, from a learned event-to-process delay, is bound to the process by tool and project path within a time window, and is released automatically.

5. Objective(s) of Invention

1. To reduce the unprotected start-up interval of a process whose creation is predictable from an earlier user action, by preparing a scheduling-priority reservation before the process exists.
2. To learn, per project, class of saved file and tool profile, the delay between an upstream event and the creation of the process, using a lightweight and explainable statistical model.
3. To act only when it is confident and beneficial: to abstain on unpredictable workflows, and to arm a reservation only when CPU contention is present and the expected benefit exceeds the expected cost.

4. To bind a reservation to the correct process by tool identity and project path, and to cover the whole process tree.
5. To ensure that reservations are time-bounded, weight-capped and released automatically, so that priority is never permanent and the impact on other processes is bounded.
6. To estimate the benefit actually obtained, through randomized holdouts, and to adapt or disable the reservation when no benefit is observed.
7. To operate entirely in user space on standard Linux interfaces, without kernel modification.

6. Working principle of the Invention

The proposed method watches registered project directories for save events and merges rapid bursts of saves into a single event. Separately, it detects the creation of processes and recognises those that belong to a build: the executable is in the project's tool profile and the working directory lies inside the project. The time from the most recent save to the first such process is a delay sample, filed under the project, the class of the saved file and the tool profile, and these samples train an exponentially weighted mean and variance.

When a new save arrives, the system checks a confidence gate consisting of enough samples, a stable delay and a good recent hit-rate. It then checks a contention and cost gate: a measure of CPU pressure must indicate contention, and the expected benefit must exceed the expected cost of a false reservation. A small random share of events that pass both gates is withheld as a holdout. For all other events, the system derives a time window around the expected delay and creates a reservation by preparing a control group with elevated CPU weight. When a matching process appears, it is moved into that group immediately, and its child processes inherit the group. If no matching process appears before the window closes, the reservation expires.

When the build ends, the group becomes empty, the elevated priority is removed, and the observed delay and build time update the model and the benefit estimate. If the workflow is unpredictable, for example manual builds at irregular times, the gate fails and no reservation is made, so the system behaves exactly like the default scheduler. The reservation is a scheduling-priority reservation (a raised relative CPU share under contention), not a guarantee of CPU capacity.

- System Architecture Diagram:

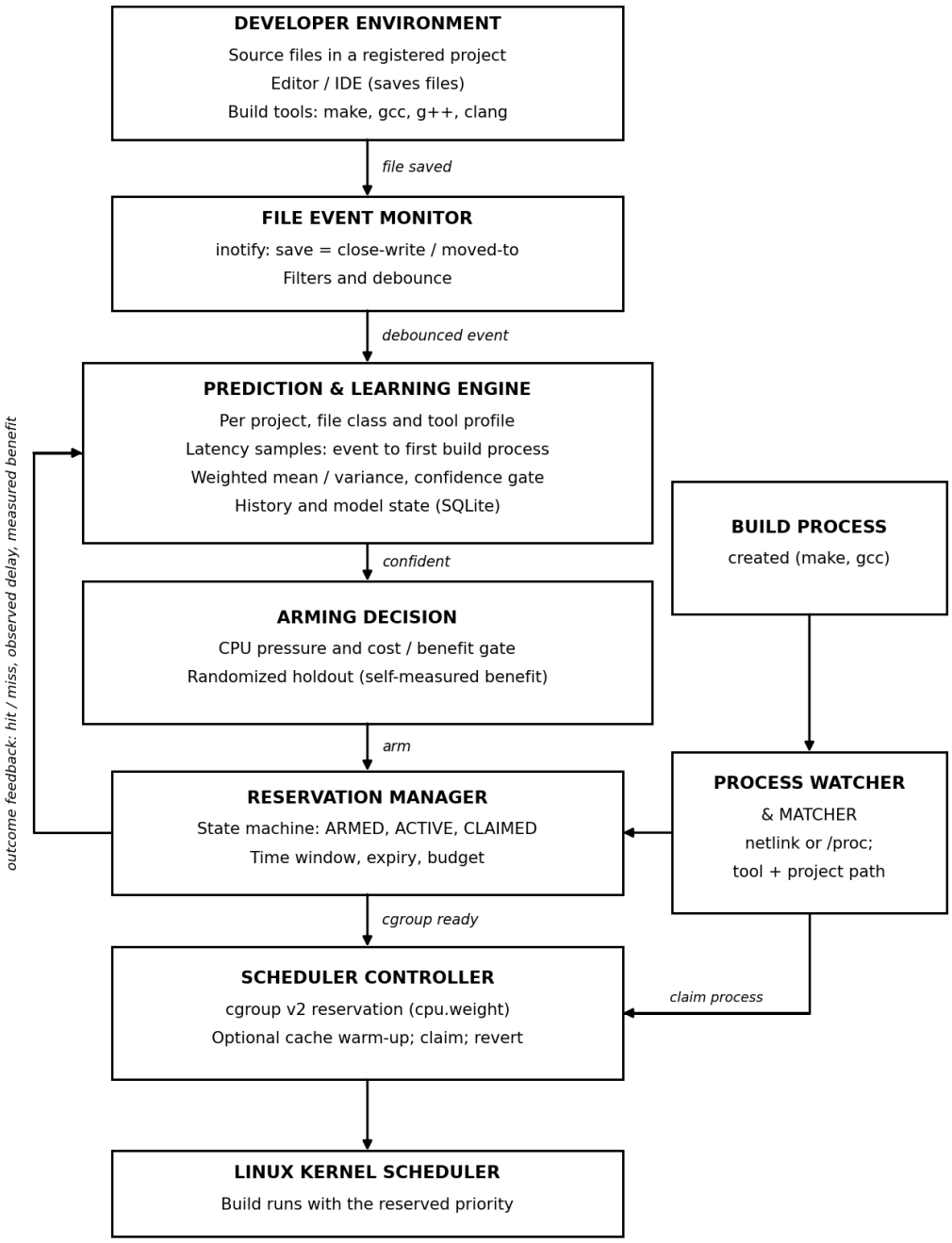


Fig. 1. Overall System Architecture for Anticipatory, Build-Aware CPU Scheduling

7. Description of the invention in detail

Stage 1: Event Detection Project roots are registered with the system. Recursive filesystem watches are placed on each project, and file-close-after-write and move-into-directory events are treated as saves, which covers both in-place and rename-based editors. Temporary files, version-control folders and build outputs are ignored, and saves within a short debounce period are merged into one event stamped at the last save. Saves are suppressed while a claimed build is running so that generated files are not mistaken for saves.

Stage 2: Process-Creation Observation and Identity Matching New processes are detected through the kernel's process-event interface, or by polling the process table where that interface is unavailable. A process matches when its executable is in the project's tool profile, its working directory (with symbolic links resolved) is inside the project root, and it is not already inside a reserved group.

Stage 3: Event-Conditioned Latency Learning For the first matching process after an event, the delay x is accepted if it lies within a maximum correlation window and is filed under the project, the class of the saved file

(for example source file or header) and the tool profile, so that different kinds of saves are modelled separately. The model is updated as $\text{mean}_n = (1 - \alpha)\cdot\text{mean}_{(n-1)} + \alpha\cdot x_n$ and $\text{var}_n = (1 - \alpha)\cdot(\text{var}_{(n-1)} + \alpha\cdot(x_n - \text{mean}_{(n-1)})^2)$, with outliers clipped once enough samples exist. A save with no matching build within the timeout is recorded as a no-build outcome.

Stage 4: Confidence Gate and Prediction Window A prediction is made only if the sample count, the coefficient of variation (standard deviation divided by mean) and the recent hit-rate pass their thresholds. The prediction window, measured from the save time, is $[\text{mean} - k\sigma - m, \text{mean} + k\sigma + m]$, where σ is the standard deviation, k a multiplier and m a safety margin. In the embodiment the model starts in observation-only mode and the gate requires at least 5 samples, a coefficient of variation of at most 0.5 (or a standard deviation of at most 0.25 s for very small means), and a hit-rate of at least 0.6 over the last 10 predictions once 5 predictions exist. The smoothing factor is 0.2, so the estimate adapts within roughly 5 to 10 builds; k is 2 and m is 0.3 s; a sample beyond three standard deviations is clipped before the update; a delay longer than a correlation window of 30 s is not treated as related to the event. The coefficient-of-variation limit keeps the window narrow relative to the expected delay, so a reservation is never held open for long on a weak signal; a no-build outcome is recorded as a miss and lowers the hit-rate; and a process that does not satisfy the identity criteria is ignored and never changes a sample or a reservation.

Stage 5: Contention-Gated, Cost-Aware Arming Before arming, the system reads a measure of CPU contention (in the embodiment, the Linux pressure stall information for the CPU) and compares the expected benefit of a reservation, derived from the learned hit-rate, the learned build duration and the benefit estimated in Stage 9, with the expected cost of a false reservation, namely reserved time in which no matching process appears and the load imposed on other processes. A reservation is armed only when contention is present or predicted and the expected benefit exceeds the expected cost.

Stage 6: Anticipatory Reservation When the gates pass, the system creates a reservation record and an empty control group. The record holds the project, the tool profile, the prediction window, the control-group path, the state and a claim flag. The control group contains no process and has the elevated CPU weight configured in advance, so it has no effect until a process is placed in it; the priority of no existing process is changed. The reservation starts in the ARMED state. At most one reservation exists per project and a small number system-wide, and a newer save resets the timer instead of creating a second reservation. Optionally, during the window, the system prepares what can be prepared before the process exists, such as warming the page cache with the expected tool binaries and the files affected by the saved file.

Stage 7: Identity-Bound Claiming A reservation in the ARMED or ACTIVE state accepts a claim. The state moves to ACTIVE when the window opens, and process detection is intensified. When a process satisfying the identity criteria appears, the claim is made by an atomic test-and-set of the claim flag held by the reservation manager, so exactly one process claims the reservation; a process that appears before the window opens is still claimed, and its delay is learned as a sample. The claimed process is written into the control group and the state becomes CLAIMED. If several candidates appear together, the top-most ancestor within the project is claimed, and any descendants it has already created are found through the parent-process map and moved as well; descendants created later inherit the group automatically. A process that matches the tool but not the project path is ignored. The state becomes CLOSED when the group is empty or the maximum lifetime is reached, and EXPIRED if the window ends with no claim.

- Timeline Diagram:

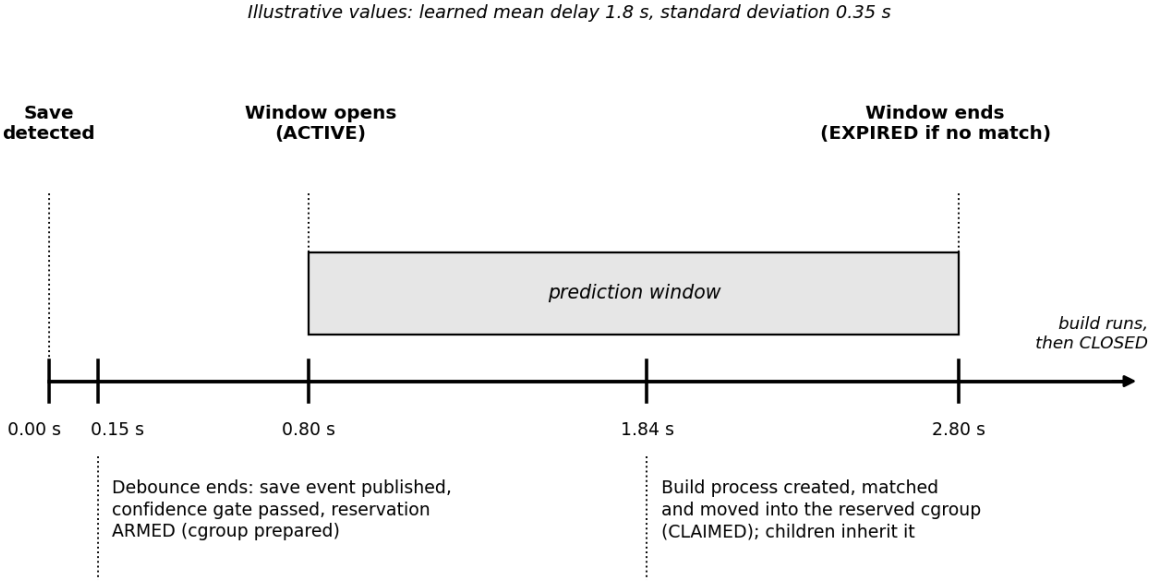


Fig. 2. Timeline of One Save-to-Build Cycle with the Prediction Window

Stage 8: Expiry, Release and Feedback The reservation lifecycle (IDLE, ARMED, ACTIVE, CLAIMED, EXPIRED, CLOSED, CANCELLED) ensures release when the window ends without a match or when the build ends. Hits, misses and observed delays feed back into the model.

- Reservation Lifecycle Diagram:

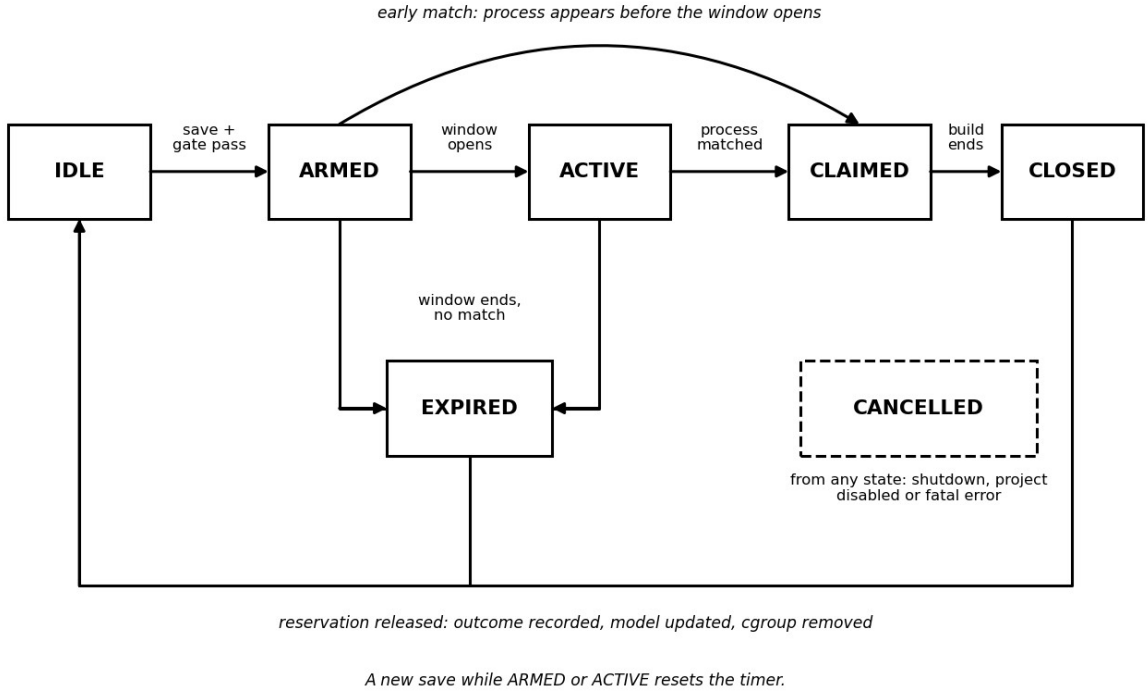


Fig. 3. Reservation Lifecycle (State Machine)

Stage 9: Self-Measured Benefit For a small, randomly selected share of events that would otherwise arm a reservation, the system withholds the reservation (a holdout) and records the build completion time and the CPU contention level. Reserved and holdout builds are compared at matched contention levels to estimate the benefit. If the estimated benefit is not significant, the system reduces the reservation weight or stops arming for that project until a benefit re-emerges.

Stage 10: Safety, Bounded Harm and Fallback Real-time scheduling classes are never used, weights are capped, and boosts have a maximum lifetime, which may be sized from the learned build duration. Optionally, a

budget limits the cumulative reserved time and weight per interval. Stale groups are cleaned at start-up, and lack of privilege degrades the system to observation-only operation.

- Method Flow Diagram:

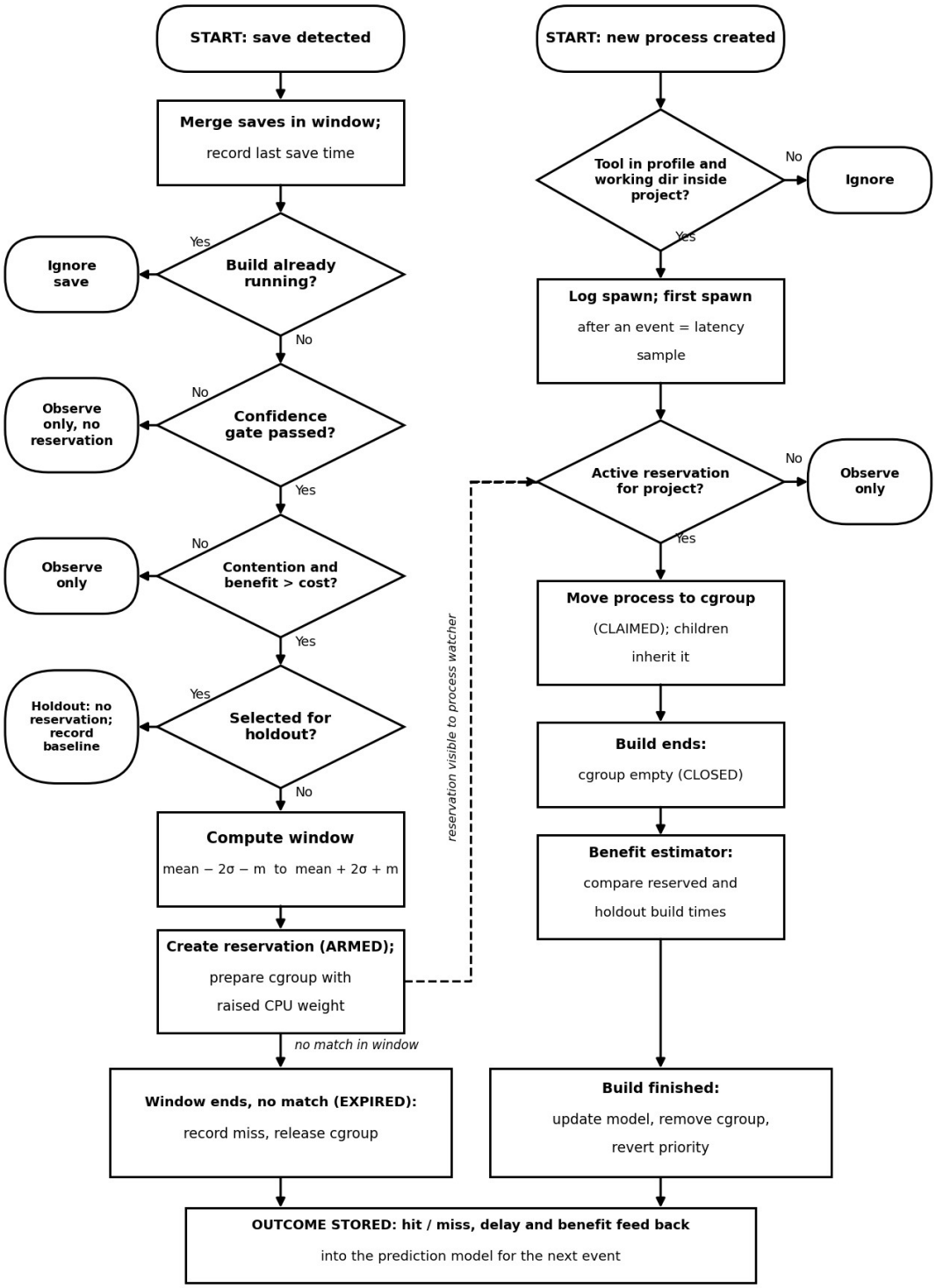


Fig. 4. Method Flow for Save Handling, Process Handling and Release

8. Experimental validation results:

Status: preliminary. The invention is at the stage of a defined design and validation protocol. Quantitative results are pending completion of the prototype tests and will be added to this disclosure as an addendum. No performance figures are claimed in this disclosure.

©VIT IPR&TTCELL

The validation is performed in a controlled environment on Linux, using registered C/C++ projects, a build-on-save script that rebuilds the changed file after a fixed delay, and a stress tool that generates background CPU load at low, moderate and high levels. It covers three trigger situations: builds that follow each save after a stable delay, builds started manually at irregular times, and saves that are not followed by any build. The emphasis is on incremental single-file rebuilds, the most frequent developer action and the case in which the start-up interval is a larger share of the total time.

Four conditions are compared: no system (C0); a reactive boost applied only after the build process is detected, with no prediction (C1); the proposed system (C2); and a static always-on boost applied to every build (C3). Five ablations are central to the validation, each compared with the full system: a reactive boost applied after process creation; the same system without the confidence gate; without the contention gate; with a fixed instead of a learned reservation window; and without the randomized holdout feedback. For each, the change in build completion time, false reservations and background-task performance is recorded, to establish whether the particular combination solves the problem better than a simpler scheduling policy.

The measurements are build completion time (median and 95th percentile), prediction hit rate, false-reservation rate, missed predictions, the time between process creation and reservation claim, the CPU overhead of monitoring and prediction, and the effect on background tasks. Each comes from at least fifty repeated, interleaved runs per condition, with more runs if the results are noisy, and is reported with confidence intervals and a non-parametric significance test.

Two further tests address the mechanisms rather than speed. A concurrency test creates several matching processes at the same moment inside an open window and checks that exactly one of them claims the reservation, with and without the atomic claim. A holdout test checks that the benefit estimated from the withheld reservations agrees with the benefit measured in the controlled comparison.

Success criteria are fixed before the runs. The technical effect is considered demonstrated if C2 shows a significantly lower median build completion time than C0 and C1 under contention, with bounded cost to other processes, and if the system makes no reservation for irregular manual builds. If prediction does not outperform the reactive boost, the contribution is assessed on precision, that is, fewer false reservations and lower impact on other processes, and the claims are adjusted accordingly. The flow below summarises the test scenario and the checks applied.

- Validation Flow:

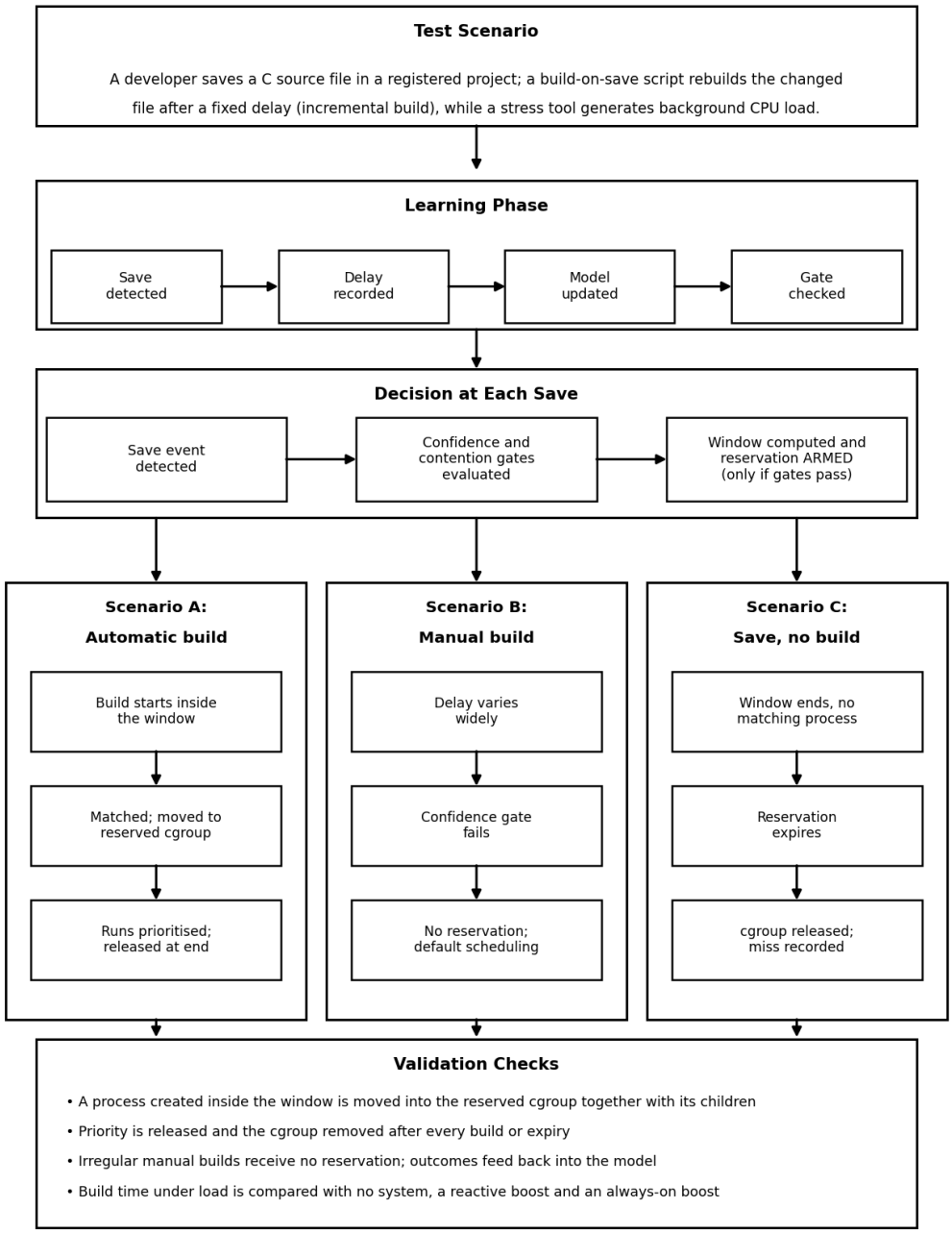


Fig. 5. Validation Flow for Anticipatory Build-Aware Scheduling

9. What aspect(s) of the invention need(s) protection?

The present invention claims protection for the following novel technical aspects. Aspects 1 to 3 together form the core inventive combination; aspects 4 to 7 are supporting features that make it reliable and measurable:

1. An event-to-process latency learning mechanism configured to learn, for a given project, class of event and tool profile, the delay between an upstream event in one subsystem (such as a file save in a monitored project) and the creation of a matching process in another subsystem, together with a measure of its variability, and to update the estimate as new observations arrive.
2. A pre-creation scheduling-priority reservation mechanism configured to create, before the predicted process exists, a time-windowed reservation comprising a reservation record and an empty control group with elevated scheduling weight, so that the arrangement is ready when the process appears. The reservation is a raised relative CPU share under contention and not a guarantee of CPU capacity.

©VIT IPR&TTCELL

3. An identity-bound claiming mechanism configured to bind the reservation, by an atomic claim, to the single process that appears within the window and matches the expected tool and project context, to extend the reservation to the process tree of that process, and to ignore processes that do not match.
4. A confidence and contention gating mechanism configured to suppress a reservation when the learned delay is unreliable (insufficient samples, high variability or low recent hit-rate), when CPU contention is absent, or when the expected benefit does not exceed the expected cost of a false reservation.
5. A reservation lifecycle and feedback mechanism configured to expire and release unused reservations automatically, to remove the elevated priority when the process ends, and to feed hits, misses and observed delays back into the learned estimate.
6. A self-measured benefit mechanism configured to withhold a random share of reservations that would otherwise be armed, to compare reserved and withheld build times at matched contention, and to reduce the reservation weight or stop arming when no benefit is observed.
7. An integrated anticipatory-scheduling architecture combining event monitoring, event-to-process latency learning, confidence and contention gating, pre-creation reservation, identity-bound claiming, automatic release and self-measured benefit for processes whose creation follows a predictable user action.

©VIT IPR&TTCELL

10. What is Technology readiness level of your invention? (Tick the appropriate TRL)

Research			**Development**			**Deployment**		
TRL 1	**TRL 2**	**TRL 3**	**TRL 4**	**TRL 5**	**TRL 6**	**TRL 7**	**TRL 8**	**TRL 9**
Basic Principles observed	Technology concept formulated	Experimental proof of concept	Technology validated in a lab	Technology validated in a relevant environment (industrially relevant in case of key enabling technologies)	Technology demonstrated in a relevant environment (industrially relevant in case of key enabling technologies)	System prototype demonstration in an operational environment	System complete and qualified	Actual system proven in an operational environment (competitive manufacturing in case of key enabling technologies, or in space)
	✔							

----------END OF THE DOCUMENT----------